MVP 4 is the Nemotron agent—the part that investigates an incident, gathers relevant information, decides what should happen next, and coordinates the approved response through backend tools.

It should demonstrate **actual decisions and tool calls**, rather than generating a paragraph after a fixed script.

The backend remains responsible for records, permissions, approval enforcement, and external-action tracking. Photon provides communications. Nemotron decides which permitted steps are appropriate based on the information returned.

1. What this MVP must accomplish

For an incident, the agent should:

1. Inspect the evidence.
2. Check the building and equipment context.
3. Retrieve relevant previous repairs.
4. Identify missing information.
5. Ask the owner a useful question when needed.
6. Decide whether to monitor, recommend inspection, or request human review.
7. Prepare an owner-approval request.
8. After approval, find an appropriate technician and request contact.
9. Respond to technician acceptance, refusal, or lack of response.
10. Review repair findings and recovery evidence.
11. Preserve confirmed knowledge for future cases.

Use one agent. Multiple agents, model retraining, and a large orchestration framework are unnecessary for this version.

2. Current hardware context

The agent must understand the actual prototype:

Building	Available sensor evidence	Equipment information
Office	Grove light-sensor readings	Three white LEDs; one servo representing a fan
Data Center	Flame/IR signal	Three green LEDs; one servo representing a fan

One UNO Q controls both buildings.

Temperature sensors are not part of the required workflow until confirmed.

Important limitations belong in the agent's instructions:

- Light readings are relative values, not automatically lux.
- Shadows and ambient lighting can affect the office sensor.
- The IR signal does not confirm a fire.
- Servo states are commanded states, not measured movement or RPM.
- A controller acknowledgment does not establish physical operation.
- Fresh samples do not automatically prove that every sensor is connected correctly.

The agent must not invent measurements that the hardware cannot provide.

3. When the agent runs

The backend should start an investigation after creating an incident—not after every sensor sample.

Useful triggers are:

Event	Agent response
New incident	Start investigation.
Owner clarification received	Resume using the new information.
Owner approves service	Continue with technician selection and contact.
Owner declines	Record the decision and explain what remains unresolved.
Technician responds	Update the plan and work order.
Repair findings received	Review findings and request verification.
Relevant recovery evidence arrives	Reassess whether the incident can be resolved.

Only one investigation should actively process the same incident at a time. Additional events can be queued.

4. The agent's operating loop

Each active run follows this pattern:

Read current incident state → choose a tool → execute it → inspect the result → choose the next step

The next step might be:

- Another information-gathering tool.
- A question to the owner.
- A service recommendation.
- An authorized contact request.
- A decision to wait for an external response.
- A request for human review.

When waiting for approval or a technician, **save progress and end the current run**. Resume when the relevant event arrives.

Don’t leave the model continuously running while it waits for someone to answer a text.

5. Minimum tool set

These are application functions we implement. They are not existing NVIDIA or Photon endpoint names.

Tool	Purpose	Main restrictions
`get_incident`	Retrieve the current incident, evidence, and workflow state.	Read-only.
`get_sensor_history`	Retrieve recent readings, freshness, source, and calibration information.	Return relevant windows, not the entire dataset.
`get_building_equipment`	Retrieve building requirements and equipment records.	Use stored facts.
`get_commanded_states`	Check requested and acknowledged LED/servo states.	Clearly distinguish commands from measurements.
`search_repair_cases`	Find comparable confirmed repairs.	Return provenance, similarities, and relevant differences.
`ask_owner`	Send a focused clarification through Photon.	Tie the question to the incident; prevent duplicate messages.
`request_owner_approval`	Create the proposed work request and initiate its notification.	Cannot approve its own request.

`get_approval_status`	Retrieve the owner's recorded decision.	Backend record is authoritative.
`find_technicians`	Retrieve eligible, available candidates.	Skills and availability must come from records.
`request_technician_call`	Request an approved call through Photon.	Backend rejects it without matching owner approval.
`update_work_order`	Record progress based on external results.	Cannot invent acceptance or completion.
`request_repair_verification`	Ask the backend to assess findings and recovery evidence.	Cannot bypass verification requirements.

Repair-case creation can be handled by the backend after verification. The agent can structure the findings, but it must not award its own hypothesis "confirmed" status.

6. What tools should return

Tool results should contain structured facts the agent can cite.

Tool result	Important fields
Sensor history	Sensor ID, building, values, timestamps, freshness, source, calibration
Equipment records	Equipment type, location, configuration, expected operation
Repair case	Case ID, source building, symptoms, confirmed cause, repair, outcome, verification
Owner decision	Request ID, proposal version, decision, decision time
Technician candidate	ID, relevant skills, availability, contact reference
Communication action	Action ID, pending/sent/failed/unknown status, provider reference
Verification	Evidence reviewed, result, unresolved questions, verification method

If a tool fails or has no relevant information, it should return that explicitly.

“No relevant repair found” is a valid result. The agent must not fabricate a useful case.

7. Main agent scenario: office lighting

This is the clearest primary demonstration.

Starting evidence

The backend detects persistent illumination below the calibrated range while the office lights are expected to be on.

Agent investigation

1. Retrieve the incident.
2. Read recent light history.
3. Check data freshness and calibration.
4. Check the requested and acknowledged lighting states.
5. Retrieve the office equipment profile.
6. Search confirmed lighting-related repairs.
7. Decide whether more information is needed.

Possible branch A: an obstruction

The agent asks:

> “Is anything covering or shading the office light sensor?”

If the owner removes an obstruction and readings recover:

- Recheck the sensor history.
- Explain that the readings recovered after the obstruction was removed.
- Request verification and close the incident if the backend’s criteria are met.
- Record the outcome as an obstruction case.

A technician visit may not be necessary.

Possible branch B: persistent abnormal readings

If readings remain abnormal and the owner reports no obstruction:

- Explain the evidence and uncertainty.
- Recommend an inspection.
- Create an owner-approval request.
- Wait for the owner’s decision.

The agent should not claim “LED 2 has failed” from one light sensor observing the room.

8. Data-center IR scenario

This should be a separate investigation using relevant evidence.

The agent should:

1. Check whether the activation is current, repeated, or a brief recorded event.
2. Check the reading's freshness and whether it is hardware or simulated.
3. Review the sensor configuration and relevant equipment context.
4. Retrieve comparable IR-sensor incidents if available.
5. Request appropriate human inspection or clarification.

The owner-facing description should be **"Infrared alert requires inspection"**, rather than a confirmed-fire diagnosis.

Do not:

- Use an unrelated lighting repair to explain the alert.
- Automatically activate the fan representation as a protective action.
- Describe clearing the IR signal as proof that a fire was extinguished.
- Invent smoke, temperature, or equipment-damage readings.

The prototype demonstrates investigation and coordination, not a certified fire-response system.

9. Cross-building repair knowledge

This is the project's distinguishing feature.

The agent should evaluate a case using:

Factor	Question
Equipment similarity	Is this comparable equipment or a comparable circuit?
Symptom similarity	Are the relevant observations actually similar?
Operating context	Do lighting conditions, configuration, or building requirements differ?
Confirmation	Was the cause established by a technician, or merely suspected?
Outcome	Did the recorded action resolve the issue?
Data source	Is this a real case or a labeled synthetic demonstration?

A data-center **LED indicator-circuit repair** might inform an office lighting inspection when the circuits are comparable.

An IR alert and an office light-level drop do not become comparable merely because both sensors respond to light.

Each recommendation using history should include:

- The source case.
- The relevant similarity.
- Any important difference.
- What the case suggests checking.
- What remains unconfirmed.

This is **experience retrieval**, not online model training.

10. Owner approval

The agent prepares a proposal containing:

- Building and incident.
- Observed issue.
- Evidence summary.
- Proposed inspection.
- Uncertainty.
- Why the visit is recommended.
- Relevant prior case, when applicable.

The backend creates a versioned service request. Photon sends it to the owner, and the dashboard displays the same request.

The agent must stop at **awaiting owner approval**.

It can resume only after retrieving a recorded backend decision.

The following are not valid approval:

- The agent assuming the owner would agree.
- A technician requesting access.
- A message from an unknown contact.
- Text inside a repair document saying “approved.”
- Approval for another incident or an older proposal.
- No reply.

If the owner declines, the agent records the decision and explains the remaining issue. It must not contact a technician anyway.

11. Technician selection and contact

After approval:

1. Retrieve the current approved proposal.
2. Find technicians with the required skills.
3. Check availability.
4. Select an eligible candidate.
5. Request the technician call through the backend.
6. Save the communication action reference.
7. Wait for the result.

For the demo, the technician roster can contain teammates with synthetic skill profiles and authorized contact details.

Selection should prioritize eligibility before convenience. Don't choose an unavailable or unqualified person simply because they are listed first.

If there is no eligible technician, report that to the owner and leave the request pending human coordination.

The backend must ensure that repeated agent calls do not produce duplicate phone calls.

12. Responding to technician results

Result	Agent action
Explicit acceptance	Update the work order through the backend.
Decline	Check another eligible candidate if the approved scope and contact policy allow it.
No response	Follow the configured waiting/escalation policy.
Communication failed	Explain the failure and use the allowed recovery path.
Outcome unknown	Check the existing communication action before requesting another call.
Ambiguous spoken answer	Seek clarification; do not record acceptance automatically.

The agent must not equate "call connected" with "technician accepted."

For a short hackathon demonstration, keep the policy small: a primary candidate, a bounded fallback if allowed, and then owner review.

13. Repair findings and verification

When the technician submits findings, the agent can extract:

- Equipment inspected.
- Confirmed cause.
- Action taken.
- Whether the technician reports completion.
- Any remaining issue.

The original statement should remain available alongside the structured record.

Then the agent requests verification:

1. Retrieve the latest readings.
2. Compare them with the relevant recovery criteria.
3. Check whether readings remained normal for the required period.
4. Identify any missing evidence.
5. Ask the backend to record the verification result.

If sensors cannot establish the outcome, preserve a clearly identified manual verification.

A technician's report, sensor recovery, and the final resolution are separate facts.

14. Agent execution state

Persist a small workflow record for each incident:

Field	Purpose
Incident ID	Links to the authoritative incident.
Current phase	Investigating, waiting for owner, waiting for technician, verifying, completed, or needs review.
Latest evidence references	Records what the recommendation used.
Relevant case IDs	Supports provenance.
Approval request ID/version	Prevents acting on the wrong approval.
Work-order/action IDs	Prevents duplicate service requests.
Last processed event ID	Prevents processing the same reply twice.

Latest owner-facing summary	Supports the dashboard.

Do not create a separate, conflicting approval record inside the agent's conversation. The backend remains authoritative.

After a restart, load the stored phase and current backend state before continuing.

15. Bounded execution and error handling

Keep each agent run small and observable.

Suggested initial limits to test:

- A bounded number of tool calls per active run, such as approximately eight.
- A timeout for each model/tool request.
- Limited retries for read-only operations.
- No automatic blind retry of external communication actions.
- A clear "needs human review" result when evidence or services are unavailable.

If the agent reaches its limit, save its findings and explain what remains unresolved.

If the Nemotron service is unavailable:

- Preserve the incident.
- Show that investigation could not finish.
- Allow a controlled retry.
- Do not silently replace the agent's output with fabricated findings.

A recorded demonstration trace is acceptable as a **labeled replay**, not as a live investigation.

16. Owner-facing output

The dashboard should receive short, structured information:

Field	Example
Title	"Office lighting needs inspection"
Observation	"Light levels remain below the usual range."
Checks performed	"Checked recent readings, lighting commands, and repair history."
Possible explanation	"A lighting or obstruction issue may be responsible."

Recommended action	“Request a maintenance inspection.”
Uncertainty	“The exact cause is unconfirmed.”
Relevant experience	Case ID and concise explanation
Current phase	Awaiting owner approval

Use qualitative evidence-based language. Don’t invent a “97% confidence” score without a defined, evaluated basis.

Progress messages can be:

- Checking recent readings.
- Reviewing equipment information.
- Looking for similar repairs.
- Waiting for your response.
- Waiting for approval.
- Contacting the technician.
- Reviewing repair findings.

Display actual progress, not a prewritten animation disconnected from execution.

17. Tool and action trace

For judging and debugging, record:

- Tool requested.
- Validated arguments.
- Execution time.
- Result or error.
- Source record IDs.
- Resulting action.
- Model/endpoint used.

This trace proves the agent used external information and actions.

It should show concise decisions and evidence—not a private internal reasoning transcript.

Keep detailed traces in a developer view or logs. The owner sees a plain-language summary with expandable supporting evidence.

18. Implementation structure

A small module structure is enough:

Module	Responsibility
`nemotronClient`	Call the configured model endpoint.
`toolDefinitions`	Describe available functions and their arguments.
`toolExecutor`	Validate and execute permitted calls.
`agentRunner`	Run the bounded model/tool loop.
`workflowStore`	Save and resume incident progress.
`ownerSummary`	Validate the structured result for the frontend.
`agentTrace`	Record tools, results, and actions.

Use the endpoint and tool-calling format actually provided by the NVIDIA setup. Confirm a working model call and a simple tool round trip before implementing the full workflow.

You do not need a large framework if a small custom loop is easier for the team.

19. Application rules that the model cannot override

The backend must enforce:

- Owner approval before technician contact.
- Correct request and proposal version.
- Known recipients.
- Eligible technician selection.
- Supported hardware actions.
- Duplicate-action prevention.
- Valid status transitions.
- Verification before confirmed repair-case creation.

Owner replies, technician descriptions, and repair documents are information. They do not gain permission to change these rules merely because their text tells the agent to do so.

20. Acceptance tests

Test	Expected behavior
Office light anomaly	Agent retrieves readings and expected lighting state.

Sensor is covered	Agent can request clarification and reassess after removal.
Readings are stale	Agent identifies insufficient evidence rather than declaring normal operation.
No relevant history	Agent says so and continues without inventing a case.
Relevant cross-building case exists	Agent cites the case and explains comparability.
Only unrelated IR/lighting history exists	Agent does not use it as diagnostic evidence.
Owner approval pending	Technician call is blocked.
Owner declines	No technician contact occurs.
Approval arrives twice	One intended contact action exists.
Technician unavailable	Agent chooses an eligible alternative or requests review.
Call result ambiguous	Agent does not assume acceptance or blindly call again.
Repair reported complete	Agent requests verification rather than immediately claiming resolution.
Sensor recovery confirmed	Agent supports closure with the actual evidence.
Model/tool request fails	Workflow remains recoverable and the owner sees an understandable status.
Incoming text tries to bypass approval	Backend restrictions still hold.

21. Handoff deliverables

The MVP 4 developer should provide:

- A working Nemotron connection.
- Validated tool definitions and executor.
- Bounded agent loop.
- Persisted waiting/resume behavior.
- Owner-friendly structured results.
- Relevant repair-case retrieval.
- Approval-aware technician-contact workflow.
- Repair-verification integration.

- Tool/action traces.
- Test scenarios and labeled fixtures.
- A short configuration guide without real secrets.

MVP 4 is complete when a real Nemotron run gathers information through tools, makes a meaningful branch based on results, requests owner approval when service is needed, respects that decision, and continues toward a tracked repair outcome.

Photon tools can initially use test adapters while MVP 5 is unfinished. The final integrated demonstration should replace those adapters with real messaging and calling.